MYTHIC

Pythia Retraining Guide

May 2026

Document Version 1.0
SDK Version v26.05.0

MYTHIC

1 Introduction

Mythic® provides the Pythia-70m language model as part of its model zoo. The training environment from https://huggingface.co/EleutherAI/pythia-70m is preserved in model-zoo so customers can leverage known tools. The steps to train the model and deploy to silicon are:

1. Convert a pre-trained FP32 PyTorch model to ONNX. This is used as a starting point for Mythic's analog-aware training. There are three available options:
   - The SDK includes a Pythia-70m model, pre-trained on the SlimPajama dataset, in ONNX format: `pythia_70m_fp32.onnx`.
   - Alternatively, you may use your own Pythia ONNX file.
2. Convert the FP32 model for analog-aware retraining.
3. Perform analog-aware retraining.
4. Verify the accuracy of the analog-aware model using a software simulation of the Mythic AMP.

2 Hardware Requirements

see YOLO Retraining Guide for Mythic AMP#Hardware-Requirements

3 Prerequisites

see YOLO Retraining Guide for Mythic AMP#Prerequisites

4 Task Configuration

1. Set the environment variable `MYTHIC_SDK_ROOT` to point to the SDK directory. For example,

```
export MYTHIC_SDK_ROOT=~/mythic_sdk/v26.05.0/
```

2. Go to the model-zoo source code directory:

```
cd $MYTHIC_SDK_ROOT/mythic-model-zoo
```

3. Activate the Python environment and the environment variables:

```
. ./scripts/pythia/pythia-m2000.env
```

The Mythic model-zoo scripts use Hydra to handle configuration files and command-line parameters. The following sections show some basic ways of overriding configuration parameters. For advanced usage, please refer to the Hydra documentation.

5 Dataset Setup

To train Pythia on your dataset, first create a set of text documents you would like to train on. Then tokenize each document using the Pythia tokenizer and store the resulting tokens in one or more binary files. The tokens should be stored in little-endian uint16, with documents separated by an EOS token.

Once this pre-tokenization step is complete, update the `train_data_dir` parameter of the `data` section in the configuration file `configs/pythia/training_config/training_config.yaml`. Here,

`train_data_dir` corresponds to the directory in which the `.bin` training files exist. Similarly, update the `val_data_path` parameter of the `data` section to point to your `.bin` validation dataset file.

6 Conversion and Retraining Steps

Each of the following steps generates an output file. Please confirm that a valid output has been generated before proceeding to the next step.

6.1 Exporting the Floating Point model to ONNX

The Mythic conversion and retraining process require as a starting point an initial floating-point ONNX model. This can by obtained by running:

```
python3 scripts/common/convert_model.py steps=export_to_onnx
```

Output: This will create a new directory under `data` named `export-with-kv-cache`, inside of which will be the floating-point ONNX model `model.onnx`. If desired, the output directory storing the `model.onnx` file can be modified by changing the `output_dir` inside the `export` section of the configuration file `configs/pythia/training_config/training_config.yaml`.

6.2 Convert the Floating Point ONNX model to Structural

A "structural" model is an intermediate model in the conversion process from floating point to retraining. Using the ONNX export obtained from the precious step, you can obtain the structural model by running:

```
python3 scripts/common/convert_model.py steps=to_structural fp_model=data/export-with-kv-cache/model.onnx
```

Output: The converted model will be saved under `data/structural.onnx`.

6.3 Convert the Structural Model to Retraining

Conversion from structural to retraining is done with the `to_training` step:

```
python3 scripts/common/convert_model.py steps=to_training
```

Output: The retraining ONNX `data/mythic.onnx`

6.4 Retrain

The `data/mythic.onnx` can be retrained in the `train` step. It uses torchrun for multi-GPU training.

```
torchrun --standalone --nproc-per-node=auto scripts/common/convert_model.py steps=train
```

Output: The trained ONNX `data/trained.onnx`

6.4.1 Hyper-parameters

Hyper-parameters (batch size, learning rate, training duration, etc.) can be specified by modifying the configuration file at `configs/pythia/training_config/training_config.yaml`.

6.5 Compute Accuracy Metrics

6.5.1 M2000

Compute detailed metrics on the retrained model with the command:

```
python3 scripts/common/convert_model.py steps=eval_trained
```

- **Output**: The accuracy metrics `data/metrics.json`.

Compute Monte Carlo-based 100PPM estimates:

```
python3 scripts/common/convert_model.py steps=mc_eval_trained
mc_eval_trained.num_samples=100
```

- **Output**: One file per sample in `data/collected_metrics/`

Print the 100PPM estimates after collecting samples:

```
python3 scripts/common/convert_model.py steps=mc_summarize_metrics
```

- **Output**: The accuracy 100PPM metrics `data/collected_metrics/metrics_summary.json`

6.6 Generate Text from the Trained Model

Generate text from the analog-aware trained model with the command

```
python3 scripts/common/convert_model.py steps=generate_text generate_text.src=/path/to/
trained_model.onnx training_config.generate.prompt="Insert your prompt here."
```

- **Output:** The generated text from the language model will be printed to stdout, starting with your input prompt.